

Unidad II

Representación de la información

ARQUITECTURA DE LA COMPUTADORAS



Como vimos en la unidad anterior, las limitaciones de la electrónica convencional imponen una sola forma práctica de representación: la lógica de dos estados:

0 y 1

Cualquier dato puede ser representado mediante una lógica binaria.

El bit es la unidad mínima de información. El problema a resolver es cómo los bits pueden representar los datos de una manera útil y cómo recuperar, posteriormente, esos datos.

Unidades de medida

8 bits = 1 byte

1024 bytes = 1 kilobyte

1024 kilobytes = 1 megabyte

1024 megabytes = 1 gigabyte

1024 gigabytes = 1 terabyte

1024 terabytes = 1 petabyte

1024 petabytes = 1 exabyte

1024 exabytes = 1 zettabyte

1024 zettabytes = 1 yottabyte

El problema a resolver es cómo los bits pueden representar los datos de una manera útil y cómo recuperar, posteriormente, esos datos.

El método que se emplea en computación es el siguiente:

- *dividir la memoria de la máquina en celdas de longitud fija y predeterminada de bits.*
- *Utilizar combinaciones de bits se pueden representar los símbolos deseados.*
- *A cada grupo de bits se le asigna una posición dentro de la memoria; es decir, una dirección.*

Esto permitirá encontrar dicho grupo cada vez que se necesite procesarlo.

Cuando representamos datos de forma tradicional y clara, necesitamos que ciertos grupos de bits indiquen, como mínimo, lo siguiente:

- *Los números del "0" al "9" (10 grupos de bits).*
- *Las letras del alfabeto (26 grupos de bits).*
- *Otros símbolos: { \$, #, &, *, @, %, ^, <, = }.*

Harán falta por lo menos 36 combinaciones de bits para representar las letras y números:

- *1 bit da dos combinaciones: {0, 1}, 21 combinaciones.*
- *2 bits dan cuatro combinaciones: {00, 01, 10, 11}, 22 combinaciones.*
- *3 bits dan ocho combinaciones: {000, 001, 010, 011, 100, 101, 110, 111}, 23 combinaciones.*

1. Grupos de 6 bits

- **1 bit** = 2 combinaciones posibles (**0** o **1**).
- **6 bits** = $2^6 = 64$ combinaciones únicas (desde **000000** hasta **111111** en binario).

¿Por qué 2^6 ?

Porque cada bit tiene 2 opciones, y se multiplican entre sí:

$$2 \times 2 \times 2 \times 2 \times 2 \times 2 = 64.$$

2. ¿Para qué se usan esas 64 combinaciones?

En sistemas de codificación (como **ASCII extendido** o **Base64**), estas 64 combinaciones se asignan a:

- **10 números** → Dígitos del **0** al **9** (10 caracteres).
- **26 letras** → Letras mayúsculas **A-Z** o minúsculas **a-z** (26 caracteres).
- **28 símbolos** → Como **+**, **-**, **@**, **#**, etc. (28 caracteres).

Total: $10 + 26 + 28 = 64$ caracteres posibles.

3. Ejemplo cotidiano: Base64

Es un sistema que usa **6 bits por carácter** para codificar datos binarios (como imágenes o archivos) en texto. Por ejemplo:

- La combinación **000000** podría representar la letra **A**.
- La combinación **111111** podría representar el símbolo **/**.

Así, cualquier dato se convierte en una cadena segura de letras, números y símbolos.

4. ¿Por qué no usar menos bits?

- Con **5 bits** ($2^5 = 32$) no alcanzaría para tantos caracteres.
- Con **6 bits** (64) se cubren justo las necesidades básicas de números, letras y símbolos.

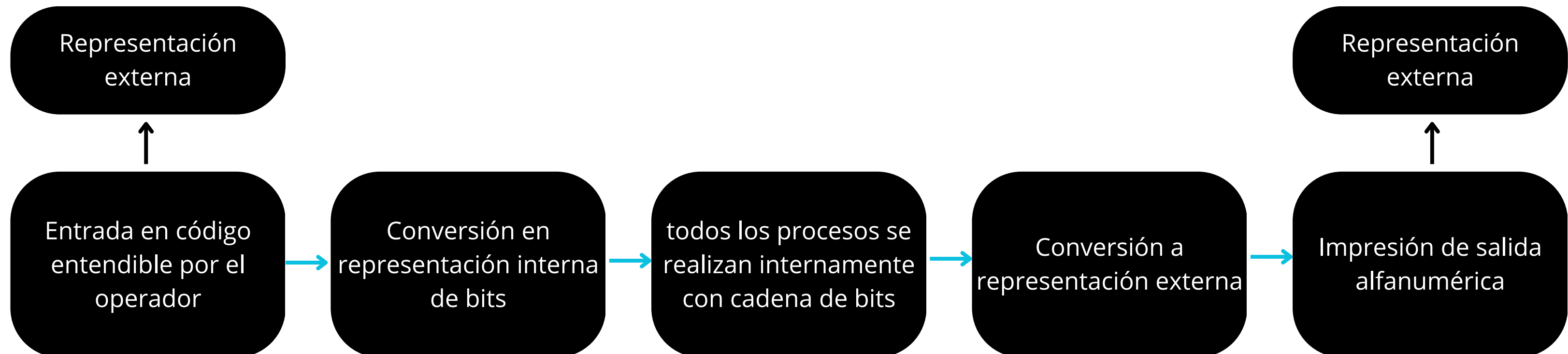
Harán falta por lo menos 36 combinaciones de bits para representar las letras y números:

- *1 bit da dos combinaciones: {0, 1},*
- *2 bits dan cuatro combinaciones: {00, 01, 10, 11},*
- *3 bits dan ocho combinaciones: {000, 001, 010, 011, 100, 101, 110, 111},*

Utilizando grupos de 6 bits se obtienen $2^6 = 64$ combinaciones posibles de bits que nos permiten representar 10 números, 26 letras, 28 símbolos.

Durante cierto tiempo se utilizaron códigos de 6 bits, a ese conjunto se le llamó carácter.

En resumen, los datos externos deben pasar primero por una conversión a una representación interna que permita su procesamiento. Posteriormente, se transforman nuevamente en una representación externa para que el usuario pueda interpretarlos con claridad.



Codificación

Aplicar un conjunto de reglas inequívocas que especifiquen la forma en que pueden representarse los datos, de forma tal que sea posible la decodificación ulterior.

- *posibilita el manejo de la información que, de otro modo, no sería posible*
- *La codificación permite representar números, letras, instrucciones y estados internos de una computadora.*
- *Existen códigos universales como ASCII, BCD y EBCDIC, entre otros. En definitiva, **codificar es establecer una correspondencia única entre una información y una configuración binaria.***

ASCII

Este código utiliza 7 bits que proporciona $2^7 = 128$ combinaciones diferentes. El octavo bit o bit más significativo, es decir, el de la izquierda, es el bit de paridad; sirve para garantizar la conservación del contenido de un byte.

- Cada dígito (0 al 9) tiene 011 como la parte de zona del byte y su representación binaria como la parte numérica.
- Los caracteres alfabéticos (mayúsculas) están distribuidos en dos grupos según que sus bytes tengan 100 o 101 como la parte de zona. Se adjunta la tabla de códigos ASCII de 7 bits; se utiliza “tal cual”, es decir, sin paridad, añadiendo un cero en la posición izquierda, o con paridad, añadiendo en esa posición el bit adecuado.

ASCII

- Cada dígito (0 al 9) tiene 0101 como la parte de zona del byte y su representación binaria como la parte numérica.
- Los caracteres alfabéticos (mayúsculas) están distribuidos en dos grupos según que sus bytes tengan 1010 o 1011 como la parte de zona.

Existe también el código ASCII - 8; este código utiliza los 8 bits para almacenar un carácter y el computador almacena un bit extra o noveno bit llamado bit de paridad.

EBCDIC

El código ampliado decimal codificado en binario hace uso de 8 bits, presente principalmente en equipos IBM y compatibles.

- Cada dígito tiene 1111 como la parte de zona del byte y su representación binaria, como la parte numérica.
- Los caracteres alfabéticos (mayúsculas) están distribuidos en tres grupos, ya sea que los bytes tengan 1100, 1101 o 1110 como bits de zona.

Representación de datos numéricos

Representar números no es una operación sencilla y es preciso diferenciar varios casos.

Existen dos métodos fundamentales para representar un dato numérico:

- codificar los dígitos decimales individualmente
- codificar el número completo

Codificación de número completo

Representación en formato de punto fijo

- El primer componente de un formato es su longitud; el segundo, es el convenio escogido en la representación o codificación del número.
- La representación en formato fijo es la manera más natural de escribir un número en una palabra de memoria.
- Para representar enteros grandes, necesitamos varios bytes, pero para hacer operaciones aritméticas con eficacia hay que emplear un número de bytes fijo, no variable; por tanto, la determinación del número de bytes supone la del mayor número representable.

Codificación de número completo

En el caso de representar numeros negativos o positivos, por convención se debe utilizar:

- El bit reservado para el signo es "0" si el número es positivo.
- El bit reservado para el signo es "1" si el número es negativo.

Forma binaria directa

Para representar números enteros puede usarse la forma binaria directa, que es la representación decimal del número en el sistema binario.

Con 8 bits pueden representarse directamente los números comprendidos entre 00000000 y 11111111

- El bit reservado para el signo es "0" si el número es positivo.
- El bit reservado para el signo es "1" si el número es negativo.

Representación de punto flotante

Es una forma en que las computadoras guardan y trabajan con números grandes, pequeños o con decimales, usando un sistema parecido a la notación científica (como "número $\times 10^{\text{potencia}}$ ").

Tres partes clave

- Un solo bit reservado a la izquierda dice si el numero es + o -
- Exponente: Controla la escala de magnitud de un numero (si es grande o chico)
- Mantisa: Contiene los dígitos importantes del numero después del punto decimal. Entre mas larga, mas preciso el numero.

Codigo ASCII

Es como un diccionario universal que usan las computadoras para convertir letras, números y símbolos en números que ellas entienden (y viceversa).

Usa 7 bits - Es decir $2^7 = 128$ combinaciones (0 al 127)

Incluye:

- Letras mayúsculas (A-Z) y minúsculas (a-z).
- Números (0-9).
- Símbolos básicos (@, #, %, etc.).
- Caracteres de control (como "Enter" o "Escape").

Codigo ASCII

Si presionamos en el teclado alguna letra, numero o carácter, el sistema interpreta un numero, lo cual devolverá, en pantalla, el dato correspondiente que solicita el usuario.

- A = 65, B = 66, ..., Z = 90.
- a = 97, b = 98, ..., z = 122.
- 0 = 48, 1 = 49, ..., 9 = 57.
- Ejemplo: Al presionar la tecla **"H"** en el teclado, la computadora no ve una letra, ve el numero **72** (gracias al ASCII).
- Todos los dispositivos (computadoras, celulares, impresoras) usan el mismo código y se entienden.

El código ASCII no se usa para caracteres chinos, árabes, emojis.

UNICODE (ASCII moderno)

¿Qué es? Un estándar que asigna un número único (code point) a cada carácter de casi todos los idiomas del mundo (incluyendo emojis, símbolos antiguos, etc.).

- 你 (chino) = U+4F60
- 안 (coreano) = U+C548
- ء (árabe) = U+0621

UTF-8 (la codificación más usada)

- ¿Cómo funciona? Es una forma práctica de guardar caracteres Unicode en la memoria de la computadora.
 - Usa 1 byte para caracteres básicos (A-Z, 0-9, símbolos ASCII).
 - Usa 2 a 4 bytes para caracteres complejos (como chinos o árabes).
- Ventaja: Es compatible con ASCII y eficiente en espacio.

UTF-16 y UTF-32

- UTF-16: Usa 2 o 4 bytes por carácter. Ideal para idiomas con muchos símbolos (como chino o japonés).
- UTF-32: Usa 4 bytes fijos por carácter. Poco eficiente, pero simple de procesar.

Código Hexadecimal

Es una forma de escribir números usando 16 símbolos (en vez de los 10 que usamos normalmente en el sistema decimal).

- Decimal (base 10): Usa 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- Hexadecimal (base 16): Usa 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

¿Para qué sirve?

Las computadoras trabajan en binario (0 y 1), pero escribir números largos en binario es complicado. El hexadecimal es un atajo más fácil para humanos:

- El número 255 en decimal:
 - Binario: 11111111 (8 dígitos).
 - Hexadecimal: FF (solo 2 dígitos).

Usos

Paletas de colores en aplicaciones

- #FF0000 = Rojo puro.
- #00FF00 = Verde puro.

1. Direcciones de memoria (en programación).
2. Códigos de errores (como los famosos "Error 0x80070005" en Windows).

